Number Guessing Game

Using C Programming

B.Tech Computer Science and Engineering

Student Name		PRN
Karan Damah	-	1262253256
Rugved Bajare	-	1262251027
Varad Khokle	-	1262253022
Dnyandeep Tiwaskar	-	1262252175
Nehit Kumar	-	1262251876

Project Overview

What is It?

A console-based number guessing game developed in C programming language. The computer generates a random number based on user-selected difficulty.

How It Works

Players guess the number within limited attempts. The game provides dynamic hints—Very Cold, Cold, Hot, or Very Hot—based on guess proximity.

Why This Project Matters

Logic Building

Develops problem-solving skills through conditional statements and loop structures.

Programming Fundamentals

Master core C concepts including random number generation and user input handling.

Interactive Design

Creates engaging user experience through feedback systems and game mechanics.

Difficulty Levels

Easy

Range: 1–10

Perfect for beginners learning the game mechanics.

Medium

Range: 1–50

Moderate challenge requiring strategic thinking.

Hard

Range: 1–100

Maximum difficulty testing advanced guessing skills.

Game Workflow

Display Menu

Show difficulty options to user

User Selection

Choose difficulty level

Generate Number

Create random number in range

Process Guess

Compare and provide hints

Determine Result

Win or lose condition

Algorithm

01

Start

Initialize game

02

Display Menu

Show difficulty options

03

Get Choice

Read user selection

04

Set Range

Define number boundaries

05

Generate Random

Create target number

06

Initialize Counter

Track guesses

07

Loop Until Win

Repeat guessing process

08

Display Result

Show win/lose message

09

Stop

End game

CODE

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main()
{
    int choice, min, max, number, guess;
    int attempts = 0, maxAttempts = 7;
    int diff, range;
    float percent;

    srand(time(0));

    printf("=====================================\n");
    printf("        NUMBER GUESSING GAME\n");
    printf("=====================================\n\n");

    printf("Select Difficulty Level:\n");
    printf("1. Easy   (Number between 1 and 10)\n");
    printf("2. Medium (Number between 1 and 50)\n");
    printf("3. Hard   (Number between 1 and 100)\n");
    printf("Enter your choice: ");
    scanf("%d", &choice);


    if (choice == 1)
    {
        min = 1;
        max = 10;
    }
    else if (choice == 2)
    {
        min = 1;
        max = 50;
    }
    else if (choice == 3)
    {
        min = 1;
        max = 100;
    }
    else
    {
        printf("\nInvalid choice! Please restart the game.\n");
        return 0;
    }

    range = max - min + 1;
    number = (rand() % range) + min;

    printf("\nI have selected a number between %d and %d.\n", min, max);
    printf("You have %d attempts to guess it.\n\n", maxAttempts);

    printf("Hint Guide:\n");
    printf("Very Cold  -> 50%% or more away from the number\n");
    printf("Cold       -> 30%% to 49%% away\n");
    printf("Hot        -> 10%% to 29%% away\n");
    printf("Very Hot   -> Less than 10%% away\n\n");

    // Guessing loop
    while (attempts < maxAttempts)
    {
        printf("Attempt %d - Enter your guess: ", attempts + 1);
        scanf("%d", &guess);

        attempts++;

        if (guess == number)
        {
            printf("\n🎉 Congratulations! You guessed the correct number.\n");
            printf("Attempts taken: %d\n", attempts);
            return 0;
        }
        else
        {
            diff = guess - number;
            if (diff < 0)
                diff = -diff;

            percent = (float)diff / range;

            if (percent >= 0.5)
                printf("Hint: Very Cold ❄\n\n");
            else if (percent >= 0.3)
                printf("Hint: Cold \n\n");
            else if (percent >= 0.1)
                printf("Hint: Hot \n\n");
            else
                printf("Hint: Very Hot \n\n");
        }
    }

    printf("You Lose!\n");
    printf("The correct number was: %d\n", number);

    return 0;
}
```

OUTPUT

```
====================================
       NUMBER GUESSING GAME
====================================

Select Difficulty Level:
1. Easy   (Number between 1 and 10)
2. Medium (Number between 1 and 50)
3. Hard   (Number between 1 and 100)
Enter your choice: 2

I have selected a number between 1 and 50.
You have 7 attempts to guess it.

Hint Guide:
Very Cold  -> 50% or more away from the number
Cold       -> 30% to 49% away
Hot        -> 10% to 29% away
Very Hot   -> Less than 10% away

Attempt 1 - Enter your guess: 28
Hint: Very Hot

Attempt 2 - Enter your guess: 38
Hint: Hot

Attempt 3 - Enter your guess: 1
Hint: Cold

Attempt 4 - Enter your guess: 25
Hint: Very Hot
```

```
Attempt 4 - Enter your guess: 25
Hint: Very Hot

Attempt 5 - Enter your guess: 22
Hint: Very Hot

Attempt 6 - Enter your guess: 20
Hint: Very Hot

Attempt 7 - Enter your guess: 21
Hint: Very Hot

You Lose!
The correct number was: 24


=== Code Execution Successful ===
```

Hint System

The game provides real-time feedback based on how close your guess is to the target number:

Very Cold

Your guess is far from the actual number. You're way off track!

Cold

Moderately far from the target. Getting warmer, but still distant.

Hot

Close to the number! You're getting very near the correct answer.

Very Hot

Extremely close! You're just a small step away from victory.

Technologies Used

C Programming

Core language for game development

stdio.h

Input/output operations

stdlib.h

Random number generation

time.h

Seed random generator

Conditional Statements

Logic control flow

Loops

Game repetition

Future Improvements

High Score System

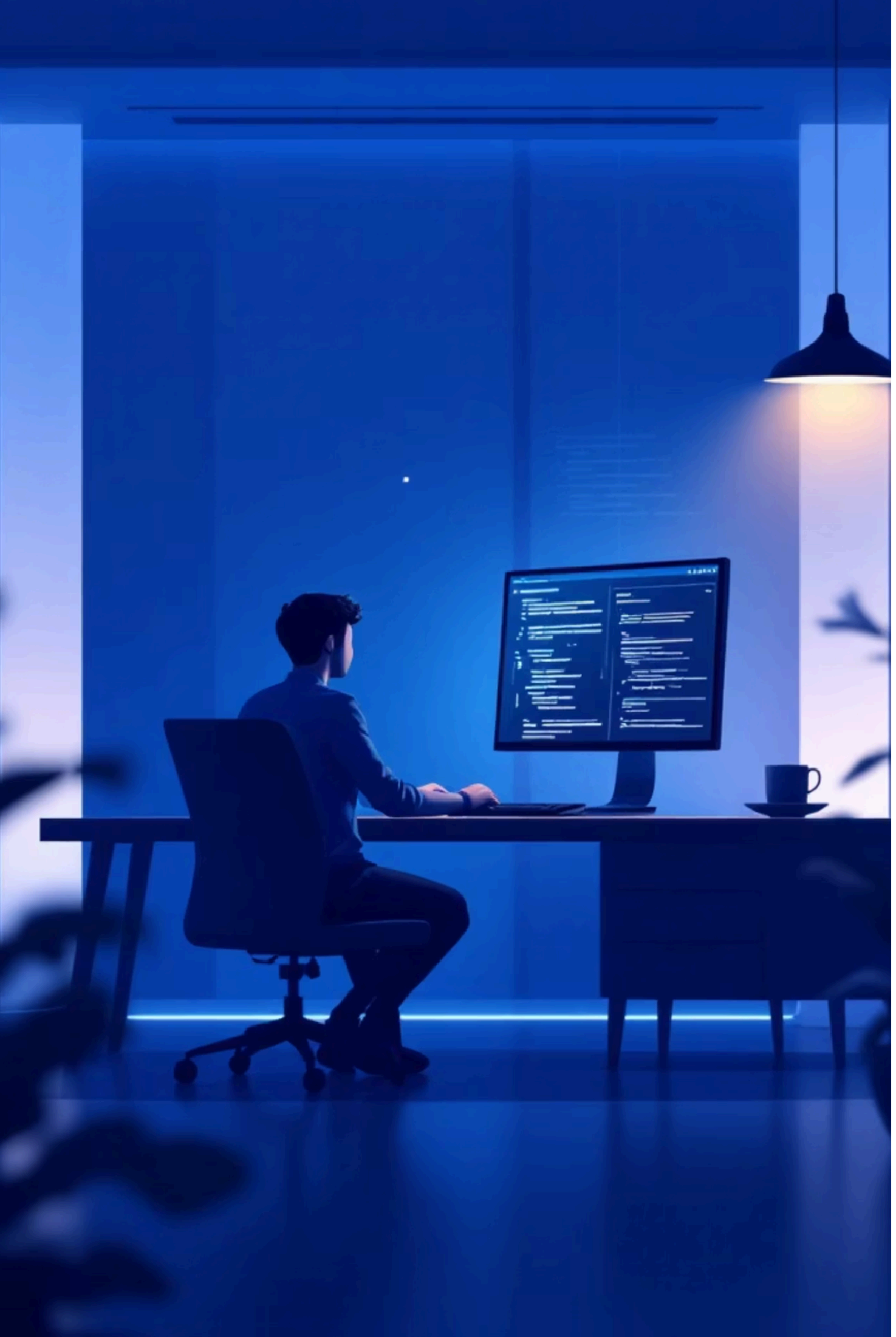
Implement file handling to store and track player scores across multiple game sessions.

GUI Version

Develop a graphical user interface using libraries like GTK or Qt for enhanced visual appeal.

Multiplayer Mode

Add competitive multiplayer functionality allowing multiple players to compete in guessing.



Key Takeaways

Strong Foundation

Mastered core C programming concepts including loops, conditionals, and random number generation.

Interactive Design

Created engaging user experience through dynamic hint system and responsive feedback.

Scalable Architecture

Built modular code structure that supports future enhancements and feature additions.

THANK

YOU